

EXPERIMENT - 41

1. Problem Statement

Write a menu-driven C program that reads two paragraphs. Use separate user-defined functions for each operation. Use call by reference by passing pointers wherever a function modifies a string or returns multiple results. Perform the following tasks:

- Read two paragraphs from the user.
- For each paragraph, identify and display the longest and shortest words.
- Remove repeated consecutive spaces from each paragraph and display the modified paragraph.
- Convert each paragraph to title case and display the modified paragraph.
- Find and display the frequency of every distinct word in each paragraph. Words must be compared without considering differences in letter case.
- Exit.

2. Algorithm

Main Program

Step 1: START

Step 2: Call readParagraph() twice to read para1 and para2.

Step 3: Display the menu and read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, display para1 and para2.

 If choice is 2, call findLongestAndShortestWords() for each paragraph and display the two words each time.

 If choice is 3, call removeRepeatedSpaces() for each paragraph and display the result.

 If choice is 4, call convertToTitleCase() for each paragraph and display the result.

 If choice is 5, call findWordFrequencies() for each paragraph.

 If choice is 6, display "Exiting the program."

Step 5: Repeat from Step 3 while choice is not equal to 6.

Step 6: STOP

Function readParagraph()

Step 1: Read a whole line into paragraph using fgets().

Step 2: If the last character is '\n', replace it with '\0'.

Step 3: Return to the calling function.

Function findLongestAndShortestWords()

Step 1: Copy paragraph into temp and set longestWord and shortestWord to empty.

Step 2: Get the first word of temp using strtok().

Step 3: Repeat while the word is not NULL:

Set `len = strlen(word)`.

If `longestWord` is empty or `len` is greater than `strlen(longestWord)`, copy word into `longestWord`.

If `shortestWord` is empty or `len` is less than `strlen(shortestWord)`, copy word into `shortestWord`.

Get the next word using `strtok()` with `NULL`.

Step 4: Return to the calling function.

Function `removeRepeatedSpaces()`

Step 1: Set `j = 0` and repeat for `i = 0` to `strlen(paragraph) - 1`:

If `paragraph[i]` is a space and the character already written at `j - 1` is also a space, skip it.

Otherwise copy `paragraph[i]` into `modifiedParagraph[j]` and add 1 to `j`.

Step 2: Put `'\0'` at `modifiedParagraph[j]`.

Step 3: Return to the calling function.

Function `convertToTitleCase()`

Step 1: Set `newWord = 1` and repeat for `i = 0` to `strlen(paragraph) - 1`:

If `paragraph[i]` is a letter, store `toupper()` of it when `newWord` is 1 and `tolower()` of it otherwise, then set `newWord = 0`.

Otherwise store the character as it is, and set `newWord = 1` if it is a space.

Step 2: Put `'\0'` at the end.

Step 3: Return to the calling function.

Function `findWordFrequencies()`

Step 1: Copy `paragraph` into `temp` and set `n = 0`.

Step 2: Get the first word of `temp` using `strtok()`.

Step 3: Repeat while the word is not `NULL`:

Convert every character of the word to lower case using `tolower()`.

Search `words[0]` to `words[n - 1]` using `strcmp()`. If it is found, add 1 to its count.

If it is not found, copy it into `words[n]`, set `count[n] = 1` and add 1 to `n`.

Get the next word using `strtok()` with `NULL`.

Step 4: Display `words[j]` and `count[j]` for `j = 0` to `n - 1`.

Step 5: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Display	Both paragraphs	Both come back exactly as typed, spaces included	Printed unchanged	PASS
2	2. Longest and shortest	Paragraph 1	programming and C	programming, C	PASS
3	2. Longest and shortest	Paragraph 2	quick and the, the first of the words of that length	quick and the, ties went to the earlier word	PASS
4	3. Remove spaces	Paragraph 1	Unchanged, it has no double spaces	Came back the same	PASS
5	3. Remove spaces	Paragraph 2	The runs of three spaces become one	Single spaces throughout	PASS
6	4. Title case	Paragraph 1	C Is A General Purpose Programming Language And C Is Very Fast.	Every word capitalised, full stop kept	PASS
7	4. Title case	Paragraph 2	The Quick Brown ... with the extra spaces still in place	Spacing untouched, only the letters changed	PASS
8	5. Frequency	Paragraph 1	c = 2 and is = 2, everything else 1	c 2, is 2, rest 1, and the two Cs merged into one entry	PASS
9	5. Frequency	Paragraph 2	the = 3, fox = 2, the rest 1	the 3, fox 2, nine distinct words in all	PASS
10	6. Exit	choice = 6	Exiting the program, and the loop ends	Loop ended	PASS

4. Source Code

paras41.c

```
/* Experiment 41. Two paragraphs are read once at the start and every
option after that works on both of them.
The one thing I had to keep remembering here is that strtok does not
just read a string, it cuts it up and writes '\0' into it. The
paragraphs come in as const, so the two functions that use strtok
take a copy first and chop the copy about instead. */

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAXPARA 500
#define MAXWORD 50
#define MAXWORDS 100

/* A word ends at a space or at punctuation, so the full stop in
"fast." is a separator and not part of the word. */
#define DELIM " \t\n.,!?:;"

void readParagraph(char paragraph[], size_t size);
void findLongestAndShortestWords(const char paragraph[], char longestWord[],
                                char shortestWord[]);
void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[]);
void convertToTitleCase(const char paragraph[], char titleCaseParagraph[]);
void findWordFrequencies(const char paragraph[]);

int main(void)
{
    char para1[MAXPARA], para2[MAXPARA];
    char longestWord[MAXWORD], shortestWord[MAXWORD];
    char modified[MAXPARA], titleCase[MAXPARA];
    int choice;

    printf("Enter the first paragraph: ");
    readParagraph(para1, MAXPARA);
    printf("Enter the second paragraph: ");
    readParagraph(para2, MAXPARA);

    do {
        printf("\nMENU\n");
        printf("1. Display the two paragraphs\n");
        printf("2. Find the longest and the shortest word\n");
        printf("3. Remove repeated consecutive spaces\n");
        printf("4. Convert to title case\n");
        printf("5. Find the frequency of every distinct word\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Paragraph 1: %s\n", para1);
                printf("Paragraph 2: %s\n", para2);
                break;

            case 2:
                findLongestAndShortestWords(para1, longestWord, shortestWord);
                printf("Paragraph 1\n");
                printf("Longest word : %s\n", longestWord);
                printf("Shortest word: %s\n", shortestWord);
                findLongestAndShortestWords(para2, longestWord, shortestWord);
                printf("Paragraph 2\n");
                printf("Longest word : %s\n", longestWord);
                printf("Shortest word: %s\n", shortestWord);
                break;

            case 3:
```

```

        removeRepeatedSpaces(para1, modified);
        printf("Paragraph 1: %s\n", modified);
        removeRepeatedSpaces(para2, modified);
        printf("Paragraph 2: %s\n", modified);
        break;

    case 4:
        convertToTitleCase(para1, titleCase);
        printf("Paragraph 1: %s\n", titleCase);
        convertToTitleCase(para2, titleCase);
        printf("Paragraph 2: %s\n", titleCase);
        break;

    case 5:
        printf("Paragraph 1\n");
        findWordFrequencies(para1);
        printf("Paragraph 2\n");
        findWordFrequencies(para2);
        break;

    case 6:
        printf("Exiting the program.\n");
        break;
    }
} while (choice != 6);

return 0;
}

/* A paragraph has spaces in it, so scanf("%s") is no use here, it would
stop at the first one. fgets takes the whole line but keeps the Enter
key at the end as a '\n', and that newline would have been counted as
a character later, so it is replaced with '\0'. */
void readParagraph(char paragraph[], size_t size)
{
    size_t len;

    if (fgets(paragraph, (int) size, stdin) == NULL) {
        paragraph[0] = '\0';
        return;
    }

    len = strlen(paragraph);
    if (len > 0 && paragraph[len - 1] == '\n')
        paragraph[len - 1] = '\0';
}

/* strtok hands back one word at a time and NULL when there are none
left. The first call is given the string and every call after that is
given NULL, which is how it remembers where it had stopped.
temp is here because strtok writes into whatever it is given, and the
paragraph is const. My first version passed the paragraph straight in
and it would not even compile. */
void findLongestAndShortestWords(const char paragraph[], char longestWord[],
                                char shortestWord[])
{
    char temp[MAXPARA];
    char *token;
    size_t len;

    strcpy(temp, paragraph);
    longestWord[0] = '\0';
    shortestWord[0] = '\0';

    token = strtok(temp, DELIM);
    while (token != NULL) {
        len = strlen(token);

        /* The empty test is for the very first word. Without it the
        comparison would be made against a string that holds nothing
        and both answers came out blank. */
        if (longestWord[0] == '\0' || len > strlen(longestWord))
            strcpy(longestWord, token);
    }
}

```

```

        if (shortestWord[0] == '\0' || len < strlen(shortestWord))
            strcpy(shortestWord, token);

        token = strtok(NULL, DELIM);
    }
}

/* A space is only dropped when the character already written before it
   was a space as well, so the first space of a run survives and the
   extra ones are thrown away. j counts the characters actually written,
   which is why it moves on its own and not with i. */
void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[])
{
    size_t i, j = 0, len;

    len = strlen(paragraph);

    for (i = 0; i < len; i++) {
        if (isspace((unsigned char) paragraph[i]) && j > 0 &&
            isspace((unsigned char) modifiedParagraph[j - 1]))
            continue;

        modifiedParagraph[j] = paragraph[i];
        j++;
    }

    modifiedParagraph[j] = '\0';
}

/* newWord remembers whether the next letter is starting a word. It is 1
   at the beginning and goes back to 1 after every space, so the first
   letter of a word is made capital and the rest are made small.
   I had forgotten the tolower at first, so SOME SHOUTING came out as
   SOME SHOUTING and not Some Shouting. */
void convertToTitleCase(const char paragraph[], char titleCaseParagraph[])
{
    size_t i, len;
    int newWord = 1;
    char ch;

    len = strlen(paragraph);

    for (i = 0; i < len; i++) {
        ch = paragraph[i];

        if (isalpha((unsigned char) ch)) {
            if (newWord)
                titleCaseParagraph[i] = (char) toupper((unsigned char) ch);
            else
                titleCaseParagraph[i] = (char) tolower((unsigned char) ch);
            newWord = 0;
        } else {
            titleCaseParagraph[i] = ch;
            if (isspace((unsigned char) ch))
                newWord = 1;
        }
    }

    titleCaseParagraph[len] = '\0';
}

/* Every word is turned into small letters before it is compared, which
   is what makes The and the count as one word. Then it is looked for in
   the list already collected. If strcmp finds it the counter goes up,
   and if it is new it is added at the end with a count of 1. */
void findWordFrequencies(const char paragraph[])
{
    char temp[MAXPARA];
    char words[MAXWORDS][MAXWORD];
    int count[MAXWORDS];
    int n = 0, j, found;
    size_t i;
    char *token;

```

```

strcpy(temp, paragraph);

token = strtok(temp, DELIM);
while (token != NULL) {
    for (i = 0; token[i] != '\0'; i++)
        token[i] = (char) tolower((unsigned char) token[i]);

    found = 0;
    for (j = 0; j < n; j++) {
        if (strcmp(words[j], token) == 0) {
            count[j]++;
            found = 1;
            break;
        }
    }

    if (!found && n < MAXWORDS) {
        strcpy(words[n], token);
        count[n] = 1;
        n++;
    }

    token = strtok(NULL, DELIM);
}

for (j = 0; j < n; j++)
    printf("%-14s %d\n", words[j], count[j]);
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra paras41.c -o paras41`. Those flags bring out the small warnings as well, and it built clean.

Reading the Two Paragraphs, Then the Longest and Shortest Words

```
adilshakil — -zsh — 159x31
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra paras41.c -o paras41
adilshakil@ADILs-MacBook-Air MyAssignment % ./paras41
Enter the first paragraph: C is a general purpose programming language and C is very fast.
Enter the second paragraph: the quick brown fox jumps over the lazy dog the fox runs

MENU
1. Display the two paragraphs
2. Find the longest and the shortest word
3. Remove repeated consecutive spaces
4. Convert to title case
5. Find the frequency of every distinct word
6. Exit
Enter your choice: 1
Paragraph 1: C is a general purpose programming language and C is very fast.
Paragraph 2: the quick brown fox jumps over the lazy dog the fox runs

MENU
1. Display the two paragraphs
2. Find the longest and the shortest word
3. Remove repeated consecutive spaces
4. Convert to title case
5. Find the frequency of every distinct word
6. Exit
Enter your choice: 2
Paragraph 1
Longest word : programming
Shortest word: C
Paragraph 2
Longest word : quick
Shortest word: the
```

Removing the Extra Spaces and Converting to Title Case

```
adilshakil — -zsh — 159x31

MENU
1. Display the two paragraphs
2. Find the longest and the shortest word
3. Remove repeated consecutive spaces
4. Convert to title case
5. Find the frequency of every distinct word
6. Exit
Enter your choice: 3
Paragraph 1: C is a general purpose programming language and C is very fast.
Paragraph 2: the quick brown fox jumps over the lazy dog the fox runs

MENU
1. Display the two paragraphs
2. Find the longest and the shortest word
3. Remove repeated consecutive spaces
4. Convert to title case
5. Find the frequency of every distinct word
6. Exit
Enter your choice: 4
Paragraph 1: C Is A General Purpose Programming Language And C Is Very Fast.
Paragraph 2: The Quick Brown Fox Jumps Over The Lazy Dog The Fox Runs
```

Word Frequencies for Both Paragraphs

```
adilshakil -- zsh -- 159x31

MENU
1. Display the two paragraphs
2. Find the longest and the shortest word
3. Remove repeated consecutive spaces
4. Convert to title case
5. Find the frequency of every distinct word
6. Exit
Enter your choice: 5
Paragraph 1
c          2
is         2
a          1
general    1
purpose    1
programming 1
language   1
and         1
very        1
fast        1
Paragraph 2
the         3
quick       1
brown       1
fox         2
jumps       1
over        1
lazy        1
dog         1
runs        1
```

The End of the Frequency List, and Exit

```
adilshakil -- zsh -- 159x31

Paragraph 2
the         3
quick       1
brown       1
fox         2
jumps       1
over        1
lazy        1
dog         1
runs        1

MENU
1. Display the two paragraphs
2. Find the longest and the shortest word
3. Remove repeated consecutive spaces
4. Convert to title case
5. Find the frequency of every distinct word
6. Exit
Enter your choice: 6
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All ten cases came out right.

The thing that caught me out first was `strtok`, and it would not even compile. `strtok` does not simply read a string, it cuts it up, writing a `'\0'` wherever it finds a separator and handing back a pointer into the middle of the original. The paragraphs arrive as `const char paragraph[]`, so passing one straight in was not allowed. Both of the functions that use `strtok` now copy the paragraph into a local `temp` first and let `strtok` chop that about, which also means the paragraph itself is never damaged and option 1 still prints it correctly afterwards.

The other `strtok` habit took a minute to get used to. The first call is given the string, and every call after it is given `NULL`, because that is how `strtok` remembers where it had stopped. I wrote the string in both places at first and it kept giving me the first word over and over.

In `findLongestAndShortestWords` I had not thought about the very first word. Without the empty test, `len` is compared against `strlen` of a string that holds nothing, so nothing is ever shorter than the shortest and both

answers came out blank. Starting both strings empty and taking the first word unconditionally fixed it.

`convertToTitleCase` was missing its `tolower` to begin with. A word typed as `SOME` came out as `SOME` rather than `Some`, because I was only making the first letter capital and leaving the rest alone. The letters after the first one have to be pushed down to lower case as well.

One decision worth mentioning. I put the full stop and the comma into the list of separators along with the space, so that `fast.` is read as the word `fast` and not as a seven letter word with a dot on the end. It matters for option 2, where the longest word would otherwise be decided by punctuation, and for option 5, where `fast` and `fast.` would have been counted as two different words.

Option 4 is the one place where the extra spaces survive on purpose. Title case only changes letters, so paragraph 2 keeps its three space gaps there, and it is option 3 that squeezes them down to one. Seeing both outputs next to each other makes it clear that the two options are doing different jobs.